

Penetration Testing

ForeGate (Prediction Market)

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test Scope	7
Security Rating	8
Severity Definitions	9
Status Definitions	10
Penetration Test Findings	11
Conclusion	25
Our Methodology	26
Disclaimers	28
About Hashlock	29

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The ForeGate team partnered with Hashlock to conduct a security Penetration Test of their ForeGate Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

ForeGate is a next-generation prediction market platform, empowering users to forecast real-world events and trade on their outcomes. Whether you're a professional trader, industry expert, or just someone who follows trends, ForeGate offers a transparent, data-driven, and engaging marketplace where your insights can translate into real earnings.

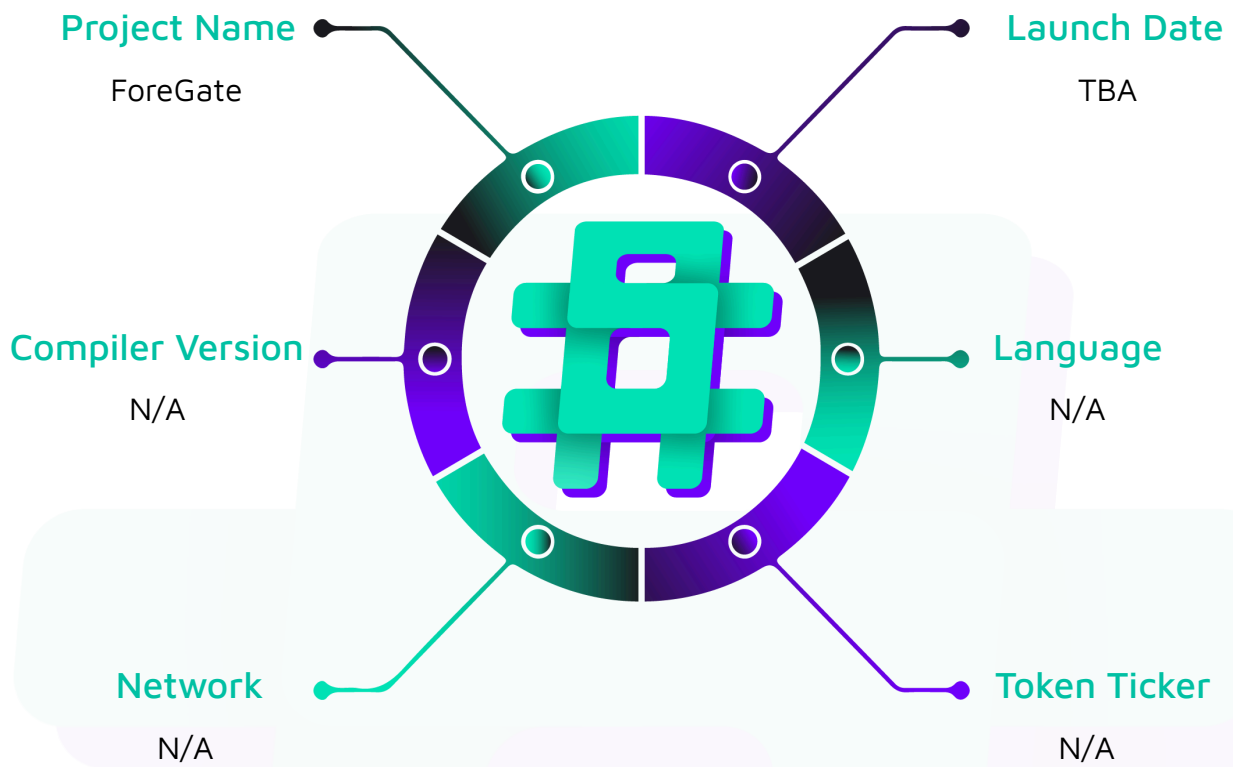
Project Name: ForeGate

Project Type: Prediction Market

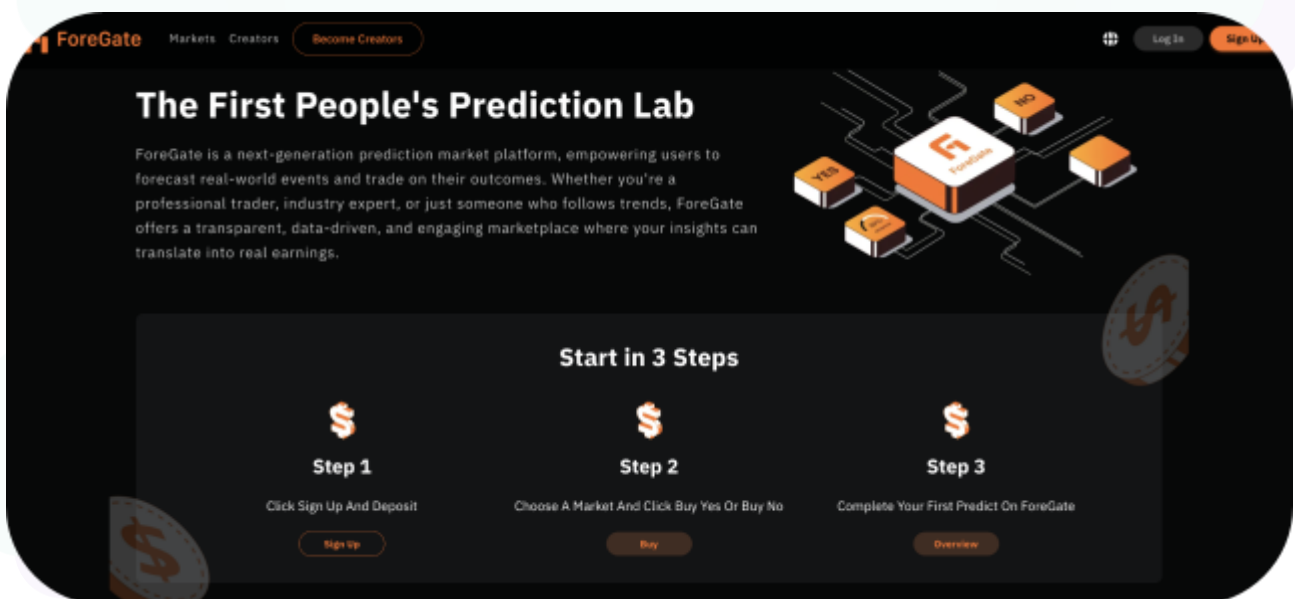
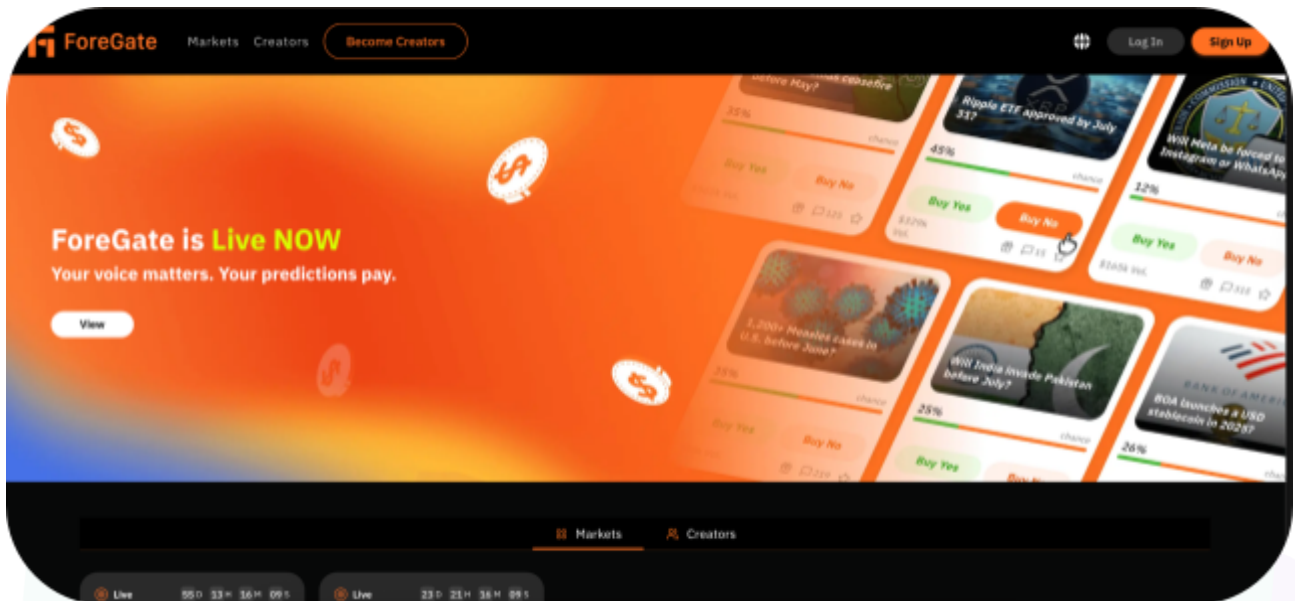
Website:

Logo:



Visualised Context:

Project Visuals:



Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the ForeGate project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	ForeGate Code Base
Platform	Web apps & APIs
Penetration Test Date	September, 2025
Repository/Website URL/IP Tested	https://github.com/qph-tech/fq-wallet/tree/uat
GitHub Commit Hash (If applicable)	7b1a99aac47c036ae9c0da08d121cab3a302f972

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Penetration Test Findings](#) section.

We have identified some vulnerabilities that need to be addressed prior to launch.

Hashlock found:

3 High-severity vulnerabilities

5 Low-severity vulnerabilities

3 QA

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

High

[H-01] WalletController - Insecure Direct Object Reference (IDOR) allows account takeover

Description

Multiple endpoints in the `WalletController` are vulnerable to Insecure Direct Object Reference. These endpoints accept a `userId` from the request body to perform sensitive actions, but fail to verify that the requesting user is authorized to act on behalf of that `userId`.

Vulnerability Details

Since the `/wallet/**` path is not protected by any authentication interceptor, these endpoints are publicly accessible. An attacker can call these functions with any known `userId` to access or modify another user's data. The `changeUserType` endpoint is particularly critical, as it allows an unauthenticated attacker to set a user's `userType` to 2 (blacklisted), effectively locking them out of the system.

`POST /wallet/changeUserType`: An attacker can provide any `userId` to change their account status, leading to a denial of service for that user.

`POST /wallet/getBalance`: An attacker can query the financial balance of any user.

`POST /wallet/getUserFundLedge`: An attacker can retrieve the complete transaction history of any user.

`POST /wallet/challengeFreeze`: An attacker can freeze the funds of any user.

Impact

This allows for unauthorized information disclosure or modification of other users' accounts.

Recommendation

Implement an authentication and authorization mechanism for all sensitive endpoints. Each request to `/wallet/**` must be authenticated, and the controller logic must verify that the identity of the authenticated user matches the `userId` being acted upon.

Note

The Foregate team informed Hashlock that the issue has been resolved internally using their own proposed fix

Status

Resolved

[H-02] Interceptors - Missing authorization on administrative endpoints

Description

The application's interceptor configuration fails to protect administrative endpoints in the `WalletNodeController`, leaving them publicly accessible.

Vulnerability Details

The `Interceptors` class in the `css` module only applies the `RequestInterceptor` to URL patterns matching `/manage/**`. The endpoints in `WalletNodeController` are mapped under `/walletNode/**`, which means they are not covered by this security constraint. As a result, endpoints like `switchNode` and `switchMode` can be invoked by any unauthenticated user.

```
// File: css/src/main/java/org/foregate/css/interceptor/Interceptors.java

@Override

public void addInterceptors(InterceptorRegistry registry) {

    registry.addInterceptor(interceptor())

        .addPathPatterns("/manage/**") // Only /manage/** is protected

    ;

}
```

Impact

An unauthenticated attacker can call `POST /walletNode/switchNode` to switch the active Solana RPC node or `POST /walletNode/switchMode` to change the block scanning mode. This could disrupt service, redirect traffic to a malicious node for transaction monitoring, or interfere with the processing of deposits and withdrawals.

Recommendation

Update the `Interceptors` configuration to apply the `RequestInterceptor` to all administrative endpoints, including `/walletNode/**`. Ensure all endpoints that can alter system state or configuration are protected by authentication and authorization.

Note

The Foregate team informed Hashlock that the issue has been resolved internally using their own proposed fix

Status

Resolved



[H-03]

[wms/src/main/java/org/foregate/wms/interceptor/IPInterceptor.java#preHandle](#) - Missing authentication and authorization enforcement

Description

The application implements `IPInterceptor` and `LoginInterceptor` to enforce request-level access control, but these mechanisms are either bypassable or completely disabled.

- Authentication bypass
 - In `IPInterceptor`, the `preHandle` function incorrectly returns `true` for non-whitelisted IPs in line 43, allowing unauthorized requests to continue.
 - Additionally, requests are allowed unconditionally if the active profile is `"dev"`. If this configuration is enabled in production by mistake, all IP-based restrictions will be ineffective.
 - In `LoginInterceptor`, all token validation logic is commented out, meaning no real authentication or session control is enforced.
- Lack of authorization checks
 - Even if authentication were working, there is no privilege separation. All authenticated users can access sensitive administrative functions.
 - For example, the `changeUserType` endpoint allows arbitrary updates of user roles, allowing attackers to escalate their account to privileged roles.
 - Another example is that users can call the `updateAssets` function to update the oracle prices arbitrarily, potentially profiting from shorting trades.

Impact

Any external attacker can directly access protected endpoints without restriction. Privileged functions (e.g., managing wallets or administrative controls) can be executed by anyone, leading to complete compromise of the system.

The attacker could also take a step further to exploit the Solana program that holds user funds, as the application controls the key of the system resolver and AMM operator.

Recommendation

Consider implementing the following recommendation:

1. Re-enable and enforce token-based authentication in `LoginInterceptor`. Remove or tightly scope any bypass tokens.
2. Modify `IPInterceptor` so that unauthorized requests return `false` and cannot proceed. Prevent "dev" bypasses from being active in production (e.g., enforce environment checks in CI/CD).
3. Implement role-based access control (RBAC) across all endpoints. Each privileged API must validate that the caller has sufficient privileges. For example, only the admin should be able to resolve pools as `AmmResolve.TYPE_SYSTEM` privilege in `wms/src/main/java/org/foregate/wms/controller/AmmController.java:302`. If regular users attempt to call this, an error should be returned.
4. Apply request throttling and enforce idempotency across endpoints to prevent abuse through repeated calls that could result in the double transfer of funds.

Note

The Foregate team informed Hashlock that the issue has been resolved internally using their own proposed fix

Status

Resolved

Low

[L-01] Multiple occurrences - Verbose error messages

Description

Multiple background scheduled jobs catch generic Exceptions and print the full stack trace to standard error using `e.printStackTrace()`, bypassing the configured logger.

While the `GlobalExceptionHandler` correctly handles exceptions for API endpoints, the background jobs' error handling behaves differently.

The use of `e.printStackTrace()` writes verbose and unstructured logs that can reveal sensitive internal information, such as library versions, file paths, and detailed logic failures, which are valuable to an attacker for reconnaissance.

Recommendation

Remove all calls to `e.printStackTrace()` and display only generic error messages to users.

Status

Resolved

[L-02] RequestInterceptor - Static authentication token for Administrative endpoints

Description

The administrative endpoints under `/manage/**` are protected by a single, static token that is loaded from a configuration file.

The RequestInterceptor checks for a token in the request header and compares it to a static value injected via `@Value("${request-token}")`. This approach means that a single token grants access to all administrative functions. Static tokens are brittle and, if compromised, provide long-lived access until manually rotated.

If the static request token is leaked from a configuration file or otherwise compromised, an attacker would gain full administrative access.

This would allow them to manually trigger any core application job, potentially leading to race conditions, double-processing of financial transactions, or other critical logic failures.

Recommendation

Replace the static token with a more secure authentication mechanism, such as OAuth 2.0 or JWTs with short expiry times.

Status

Resolved

[L-03] RequestInterceptor - Commented out IP whitelist logic

Description

The `RequestInterceptor` in the `decrypt` module contains commented-out code that was intended to provide IP address whitelisting. This feature is currently disabled.

The code to check the request's source IP against a whitelist (`ipList`) has been commented out, leaving only the static token check as the primary security control for the highly sensitive `getPrivateKey` endpoint.

While not a direct vulnerability, it removes a security layer that could have prevented unauthorized access even if the static token were compromised.

Recommendation

Re-enable the IP whitelisting functionality as an additional layer of security for the `decrypt` service.

Status

Resolved

[L-04]**common/src/main/java/org/foregate/common/amm/AmmProgram.java** -

Inconsistent and unnecessary writable flags in client instruction builders

Description

Several client-side instruction builders mark accounts as writable even though the corresponding on-chain program logic does not require them to be mutated:

- `claimSystemUnresolved` → authority marked writable even though it is not required (`cfmm-prediction-market/src/lib.rs:1753-1758`).
- `claimCreatorUnresolved` → authority marked writable even though it is not required (`cfmm-prediction-market/src/lib.rs:1786-1790`).
- `systemResolvePool` → authority marked readonly, but the documentation (`cfmm-prediction-market/DOCS.md:286`) states it should be writable.
- `setResolveStatus` → operator marked writable even though it is not required (`cfmm-prediction-market/src/lib.rs:1828-1833`).

Impact

This results in both false positives (accounts flagged as writable when not required) and false negatives (accounts not flagged as writable despite meeting spec/docs expectations).

Recommendation

Align client instruction builders with actual on-chain requirements and project documentation.

Status

Resolved

[L-05]

[common/src/main/java/org/foregate/common/utils/HttpClient.java#doPost](#)

- Missing HTTP status validation

Description

The method retrieves the HTTP response entity without validating the status code. This is because the status code validation is commented out.

Impact

Invalid requests (e.g., 401 or 500 responses) will still be parsed, which may return misleading data.

Recommendation

Uncomment the validation to enforce status code validation.

Status

Resolved

QA

[Q-01]

`common/src/main/java/org/foregate/common/amm/AmmUtil.java#getValueFromJson1` - Incorrect error logged

Description

The error log incorrectly references the method name `getValueFromJson` instead of the actual method name `getValueFromJson1`.

Recommendation

Update the log message to match the method name.

Status

Resolved

[Q-02]

`common/src/main/java/org/foregate/common/solana/program/SystemProgram.java` - Typo in variable names

Description

Two methods contain spelling errors in parameter names:

- `transfer` → `toPublikey` instead of `toPublicKey`.
- `createAccount` → `newAccountPublikey` instead of `newAccountPublicKey`.

Recommendation

Update the parameter names.

Status

Resolved

[Q-03]

[css/src/main/java/org/foregate/css/controller/WalletNodeController.java#switchMode](#) - Incomplete implementation with TODO comment

Description

The `switchMode` endpoint contains a `//todo` marker indicating unfinished logic.

Closing WebSocket connections and reinitializing Redis are only noted as comments and not implemented.

Recommendation

Implement the missing logic.

Status

Resolved

Conclusion

After Hashlock's analysis, the ForeGate project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.